

Ejercicios SQL - JOIN, GROUP BY, funciones agregadas y HAVING

Santiago Peñaranda Mejía

2025

Instrucciones: A continuación, se presenta una base de datos ficticia con 3 tablas. Lee atentamente la estructura de cada tabla y responde a las preguntas formulando consultas SQL correctas. No es necesario ejecutar las consultas: este ejercicio es para ser resuelto en papel.

Tabla clientes

Campo	Tipo	Descripción
idcliente	INT	Clave primaria
nombre	VARCHAR	Nombre del cliente
ciudad	VARCHAR	Ciudad de residencia

Tabla productos

Campo	Tipo	Descripción
idproducto	INT	Clave primaria
nombre	VARCHAR	Nombre del producto
precio	DECIMAL	Precio unitario

Tabla pedidos

Campo	Tipo	Descripción
idpedido	INT	Clave primaria
fecha	DATE	Fecha del pedido
idcliente	INT	Clave foránea a clientes

idproducto	INT	Clave foránea a productos
------------	-----	---------------------------

Parte 0: **Creación de tablas**

Escribe las sentencias SQL necesarias para crear las siguientes tablas, incluyendo claves primarias y claves foráneas donde corresponda:

1. Crear la tabla clientes con los campos idcliente (clave primaria), nombre y ciudad.

```
mysql> CREATE TABLE clientes(  
  -> idcliente INT PRIMARY KEY AUTO_INCREMENT,  
  -> nombre VARCHAR(100),  
  -> ciudad VARCHAR(100));  
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> SHOW COLUMNS FROM clientes  
-> ;
```

Field	Type	Null	Key	Default	Extra
idcliente	int	NO	PRI	NULL	auto_increment
nombre	varchar(100)	YES		NULL	
ciudad	varchar(100)	YES		NULL	

3 rows in set (0.01 sec)

2. Crear la tabla productos con los campos idproducto (clave primaria), nombre y precio.

```
mysql> CREATE TABLE productos(  
  -> idproductos INT PRIMARY KEY AUTO_INCREMENT,  
  -> nombre VARCHAR(100),  
  -> precio DECIMAL(10,2) UNIQUE);  
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> SHOW COLUMNS FROM productos;
```

Field	Type	Null	Key	Default	Extra
idproductos	int	NO	PRI	NULL	auto_increment
nombre	varchar(100)	YES		NULL	
precio	decimal(10,2)	YES	UNI	NULL	

3 rows in set (0.00 sec)

3. Crear la tabla pedidos con los campos idpedido (clave primaria), fecha, idcliente (clave foránea) y idproducto (clave foránea).

```
mysql> CREATE TABLE pedidos(
  -> idpedidos INT PRIMARY KEY AUTO_INCREMENT,
  -> fecha DATE,
  -> idcliente INT,
  -> idproducto INT,
  -> FOREIGN KEY (idcliente) REFERENCES clientes(idcliente),
  -> FOREIGN KEY (idproducto) REFERENCES productos(idproductos));
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> SHOW COLUMNS FROM pedidos;
+-----+-----+-----+-----+-----+-----+
| Field      | Type | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| idpedidos  | int  | NO   | PRI | NULL    | auto_increment |
| fecha      | date | YES  |     | NULL    |                 |
| idcliente  | int  | YES  | MUL | NULL    |                 |
| idproducto | int  | YES  | MUL | NULL    |                 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Parte 1: Consultas básicas con JOIN

- Muestra el nombre del cliente, el producto que compró y la fecha del pedido.

```
mysql> SELECT
  -> clientes.nombre AS nombre_de_clientes,
  -> productos.nombre AS nombre_productos,
  -> pedidos.fecha AS fecha_productos
  -> FROM pedidos
  -> JOIN clientes ON pedidos.idcliente = clientes.idcliente
  -> JOIN productos ON pedidos.idproducto = productos.idproducto;
Empty set (0.00 sec)
```

- Lista todos los productos comprados por clientes de la ciudad de "Madrid".

```
mysql> SELECT productos.nombre
  -> FROM pedidos
  -> JOIN clientes ON pedidos.idcliente = clientes.idcliente
  -> JOIN productos ON pedidos.idproducto = productos.idproducto;
Empty set (0.00 sec)
```

- Muestra los nombres de los productos que han sido pedidos más de una vez (sin usar GROUP BY por ahora).

```
mysql> SELECT productos.nombre
  -> FROM pedidos
  -> JOIN productos ON pedidos.idproducto = productos.idproducto
  -> WHERE productos.idproducto IN (
  -> SELECT idproducto
  -> FROM pedidos
  -> GROUP BY idproducto
  -> HAVING COUNT(*)>1);
Empty set (0.00 sec)
```

Parte 2: *Consultas con funciones agregadas y GROUP BY*

- Muestra cuántos pedidos ha hecho cada cliente.

```
mysql> SELECT clientes.nombre, COUNT(*) AS cantidad_pedidos
-> FROM pedidos
-> JOIN clientes ON pedidos.idcliente = clientes.idcliente
-> GROUP BY clientes.nombre;
Empty set (0.00 sec)
```

- Muestra cuántas veces ha sido comprado cada producto y su precio.

```
mysql> SELECT productos.nombre, COUNT(*) AS veces_comprado, productos.precio
-> FROM pedidos
-> JOIN productos ON pedidos.idproducto = productos.idproducto
-> GROUP BY productos.nombre, productos.precio;
Empty set (0.00 sec)
```

- Muestra el total ingresado por cada producto (precio × cantidad de veces vendido).

```
mysql> SELECT productos.nombre, COUNT(*) AS cantidad, productos.precio,
-> COUNT(*) * productos.precio AS total_ingresado
-> FROM pedidos
-> JOIN productos ON pedidos.idproducto = productos.idproducto
-> GROUP BY productos.nombre, productos.precio;
Empty set (0.00 sec)
```

Parte 3: *Consultas con HAVING y filtros*

- Muestra los clientes que han hecho más de 3 pedidos.

```
mysql> SELECT clientes.nombre, COUNT(*) AS pedidos
-> FROM pedidos
-> JOIN clientes ON pedidos.idcliente = clientes.idcliente
-> GROUP BY clientes.nombre
-> HAVING COUNT(*) > 3;
Empty set (0.00 sec)
```

- Muestra los productos que han generado más de 100 euros en total.

```
mysql> SELECT productos.nombre, COUNT(*) * productos.precio AS total
-> FROM pedidos
-> JOIN productos ON pedidos.idproducto = productos.idproducto
-> GROUP BY productos.nombre, productos.precio
-> HAVING total > 100;
Empty set (0.00 sec)
```

- Muestra el precio medio de los productos vendidos por ciudad.

```
mysql> SELECT clientes.ciudad, AVG(productos.precio) AS precio_medio
-> FROM pedidos
-> JOIN clientes ON pedidos.idcliente = clientes.idcliente
-> JOIN productos ON pedidos.idproducto = productos.idproducto
-> GROUP BY clientes.ciudad;
Empty set (0.00 sec)
```

Parte 4: Consulta final integradora

Muestra por cada ciudad el nombre del cliente que ha hecho más pedidos (puedes usar GROUP BY, funciones agregadas y HAVING).